Chapter 5

Raw Mass Spectrometry Data Ingestion and Backend Architecture with Spectra

Lucas VHH TRAN[1]

Abstract

Raw mass spectrometry data files must be parsed accurately before downstream identification, feature extraction, or quantification can occur. Proprietary vendor file formats store spectral arrays in closed binary containers that restrict cross-platform computing and require specialized converter engines. The HUPO Proteomics Standards Initiative established open XML schemas (`mzML`*) and controlled vocabularies (*`PSI-MS`*) to standardize scan-level metadata and binary peak representations [1, 3]. In R 4.6.1 and Bioconductor 3.20 [12, 11], the* `mzR` *package provides low-level C++ file handles via Rcpp [13] and ProteoWizard [4] that read headers on-demand without memory overhead. The modern* `Spectra` *container builds on this architecture [9, 10], offering a modular backend system that separates data manipulation logic from physical storage. Using on-disk backends (*`MsBackendMzR` *or* `MsBackendHdf5Peaks` *[5, 14]), large analytical cohorts can be manipulated with minimal RAM footprints, preserving FAIR principles [17, 18] and avoiding out-of-memory crashes while enabling lazy execution queues for peak cleaning and extraction. This chapter details the physical architecture of mass spectrometry files, the mechanics of centroiding conversion via ProteoWizard* `msconvert` *[4, 7], low-level handle seeking in* `mzR`*, and high-level stream processing in* `Spectra`*.*

Keywords: Raw mass spectrometry, mzML, ProteoWizard, msconvert, Spectra, MsBackendMzR, mzR, Lazy evaluation, On-disk processing.

Chapter Overview and Architectural Roadmap

Table 5.1 outlines the methodological and computational components required to import and audit raw mass spectrometry datasets.

Methodological Note

Tutor Warning: The Hazard of In-Memory Instantiation

Standard R data frames store all numeric vectors directly in random-access memory (RAM). Loading

[1]CK-CARE Davos, Christine Kühne--Center for Allergy Research and Education, Davos, Switzerland. Email: `lucas.tran@ck-care.ch`

Table 5.1: Architectural roadmap for mass spectrometry raw data ingestion.

Stage	Core Operation	Algorithmic Mechanism	Bioc / Tool
1. Conversion	Proprietary to open format [1]	Vendor DLL centroiding, zlib compression	`msconvert` [4]
2. Low-Level Handle	Random access scan header	C++ ProteoWizard parser, $O(1)$ seek [13]	`mzR`
3. On-Disk Abstraction	Lazy spectral streaming	Pointer mapping, zero RAM peak load [5]	`MsBackendMzR` [9]
4. Trace Extraction	Chromatographic extraction	Summed/base peak intensity against RT	`Spectra::tic` [10]
5. Queue Processing	Noise removal and filtering	Lazy processing queue, transient chunking	`filterIntensity` [8]

raw mass spectrometry peak arrays across a 50-file cohort into RAM demands over 500 GB of memory, terminating standard workstations. The modern Bioconductor `Spectra` architecture resolves this bottleneck through on-disk backends (`MsBackendMzR`). Binary peak arrays remain on storage media until explicitly queried, enabling analysis of multi-gigabyte cohorts on standard laptop hardware.

5.1 The Mass Spectrometry Data Ecosystem

Mass spectrometry instruments generate high-dimensional time-series data consisting of mass-to-charge ratios (m/z), signal intensities, and detector metadata. Historically, instrument vendors developed isolated binary formats tailored to proprietary acquisition software. Thermo Fisher Scientific instruments write single binary `.RAW` files; Sciex systems output paired `.WIFF` and `.wiff.scan` files; Agilent and Bruker utilize `.D` multi-file directory structures; and Waters relies on raw folder containers.

These closed formats prevent transparent inspection and cross-platform high-performance computing on Linux clusters. To resolve vendor fragmentation, the Human Proteome Organization Proteomics Standards Initiative (HUPO-PSI) developed open, standardized XML-based formats:

1. **mzXML:** An early XML format developed by the Seattle Proteome Center to standardize data exchange for the Trans-Proteomic Pipeline [2].

2. **mzML:** The definitive community standard established in 2008 by HUPO-PSI, merging mzData and mzXML [1, 3]. It utilizes the PSI-MS Controlled Vocabulary to encode instrument parameters, scan configurations, and data arrays unambiguously.

3. **mzMLb:** A modern container that embeds the mzML XML metadata document alongside native HDF5 binary datasets, achieving high-speed chunked I/O and lossless compression [5].

4. **imzML:** A dual-file architecture for mass spectrometry imaging, separating spatial XML metadata from high-throughput imaging binary data (`.ibd`) [6].

5.2 Anatomy of the Mass Spectrum: Profile versus Centroid

A mass spectrometer detects ion beams as continuous analog voltages or discrete digital ion-arrival events across the detector surface. The resulting signal can be stored in two distinct states:

- **Profile Mode:** The continuous ion current is recorded across multiple discrete m/z sampling channels per peak. A single peptide isotope peak is defined by 10--25 data points forming a Gaussian or Lorentzian distribution. Profile data preserves full peak width, asymmetry, and baseline noise statistics. However, file sizes range from 1 to 10 GB per LC-MS run.
- **Centroid Mode:** A mathematical peak-detection algorithm identifies the center of gravity and integrated area of each profile envelope, reducing it to a single discrete $(m/z, \text{intensity})$ pair. Centroiding reduces raw data volume by 15- to 20-fold.

Most downstream search engines (Mascot, Comet, MS-GF+) and feature-detection algorithms (`centWave` in `xcms` [15, 16]) require centroided peak lists. Applying an external, open-source centroiding algorithm to profile data after conversion often degrades mass accuracy. Conversion tools must employ vendor-native algorithms (via dynamically linked libraries on Windows or Wine) during file generation.

5.3 Data Conversion Engineering with ProteoWizard msconvert

The open-source ProteoWizard project provides the industry-standard converter `msconvert` [4], complemented in cross-platform pipelines by ThermoRawFileParser [7]. When converting vendor raw files, the order of command-line filters is critical:

```
msconvert.exe "input_sample.raw" ^
    --filter "peakPicking␣vendor␣true␣1-" ^
    --filter "threshold␣bpi-relative␣0.005␣most-intense" ^
    --32 ^
    --zlib ^
    --mzML ^
    -o "processed_mzML/"
```

Critical Filter Rules:

1. **Filter Ordering:** The `peakPicking vendor true 1-` filter must be executed first. Vendor libraries require access to original unthresholded profile arrays. Placing a noise threshold before peak picking breaks the vendor DLL interface.
2. **Precision and Compression:** Specifying `-32` writes intensities as 32-bit floating-point numbers rather than 64-bit doubles, halving array size without sacrificing analytical resolution. Specifying `-zlib` compresses binary arrays before Base64 encoding.
3. **Index Generation:** The resulting file is formatted as `<indexedmzML>`, embedding a byte-offset index table at the end of the file that enables $O(1)$ random seek operations.

5.4 Low-Level Handle Architecture: C++ Random Access with mzR

The Bioconductor package `mzR` provides a C++ interface to ProteoWizard libraries via Rcpp [13, 4]. It separates metadata header extraction from binary peak loading. Opening a file via `mzR::openMSfile()` creates

a lightweight file pointer without reading spectral arrays into R:

$$\text{RAM}_{\text{handle}} = O(1) \ll \text{File Size} \tag{5.1}$$

Calling `header(ms)` queries the binary seek table, returning an annotated data frame where each row corresponds to an individual acquisition event. Binary spectral arrays are retrieved on-demand via `peaks(ms, scan_index)`, which uses C++ `fseek()` to jump directly to the target byte offset.

5.5 Modern High-Level Container: The Spectra Architecture

The `Spectra` package modernizes mass spectrometry data analysis in R [9, 10], advancing the infrastructure established by `MSnbase` [8]. It separates user-facing methods from physical storage mechanisms via interchangeable backends:

- `MsBackendMzR`: Stores scan metadata in memory while reading peak arrays on-the-fly from original mzML, mzXML, or CDF files.
- `MsBackendMemory`: Holds all metadata and peak arrays in memory as nested lists, optimizing speed for small datasets at the cost of high RAM consumption.
- `MsBackendHdf5Peaks`: Stores peak arrays in structured HDF5 binary files on disk via `rhdf5` [14], supporting multi-gigabyte cohorts with fast chunked retrieval.
- `MsBackendSql`: Maps spectral records into a relational database (PostgreSQL, SQLite), facilitating multi-user laboratory data management.

5.5.1 Lazy Evaluation and Processing Queues

Modifications to spectral arrays (such as peak filtering, thresholding, and smoothing) do not immediately alter the underlying files or create duplicate memory arrays. Instead, `Spectra` records these operations in a *processing queue*. When peaks are requested for visualization or calculation, the queue functions are executed dynamically on the transient data chunk:

$$\text{Peaks}_{\text{out}} = (f_k \circ \cdots \circ f_2 \circ f_1)(\text{Peaks}_{\text{raw}}) \tag{5.2}$$

5.6 Laboratory Workflow and Executable Implementations

The laboratory exercises demonstrate raw data ingestion, handle inspection, memory benchmarking, and chromatographic extraction using a validated Thermo Orbitrap Fusion dataset (`MS3TMT11.mzML`) from the `msdata` Bioconductor package.

5.6.1 Listing 5.1: Low-Level Handle Inspection via mzR

R Lab Listing 1: Listing 5.1: Ingesting Scan Headers without Peak Overhead [R 4.4 / Bioconductor]

```
library(Spectra)
library(mzR)
library(msdata)
library(tibble)
library(dplyr)
```

```
f_mzml <- system.file("proteomics", "MS3TMT11.mzML", package = "msdata")
f_info <- file.info(f_mzml)
cat(sprintf("File: %s | Size: %.2f MB\n", basename(f_mzml), f_info$size / (1024^2)))

# Open file handle (zero peak data loaded)
ms_handle <- mzR::openMSfile(f_mzml)
cat(sprintf("Instrument: %s | Ionisation: %s\n",
            mzR::instrumentInfo(ms_handle)$model,
            mzR::instrumentInfo(ms_handle)$ionisation))

# Extract header table
hd <- mzR::header(ms_handle)
cat(sprintf("Total Scans: %d\n", nrow(hd)))
print(table(hd$msLevel))

# Close file handle
mzR::close(ms_handle)
```

```
File: MS3TMT11.mzML | Size: 15.82 MB Instrument: Orbitrap Fusion | Ionisation: nanoelectrospray Total Scans: 994
1 2 3 30 482 482
```

5.6.2 Listing 5.2: Memory Benchmarking across Spectra Backends

R Lab Listing 2: Listing 5.2: On-Disk vs In-Memory RAM Allocation [R 4.4 / Bioconductor]

```
# Instantiate on-disk backend
sps_ondisk <- Spectra(f_mzml, backend = MsBackendMzR())
size_ondisk <- as.numeric(object.size(sps_ondisk))

# Convert to in-memory backend
sps_inmem <- setBackend(sps_ondisk, MsBackendMemory())
size_inmem <- as.numeric(object.size(sps_inmem))

cat(sprintf("On-Disk RAM:   %.2f KB (%d bytes)\n", size_ondisk / 1024, size_ondisk))
cat(sprintf("In-Memory RAM: %.2f KB (%d bytes)\n", size_inmem / 1024, size_inmem))
cat(sprintf("Inflation Ratio: %.2fx\n", size_inmem / size_ondisk))
```

```
On-Disk RAM: 479.99 KB (491512 bytes) In-Memory RAM: 10295.07 KB (10542152 bytes) Inflation Ratio: 21.45x
```

5.6.3 Listing 5.3: Survey Scan Extraction and Base Peak Chromatogram

R Lab Listing 3: Listing 5.3: Reconstructing BPC and TIC from Survey Scans [R 4.4 / Bioconductor]

```
# Filter to MS1 survey scans
sps_ms1 <- filterMsLevel(sps_ondisk, msLevel = 1L)

# Extract chromatographic variables
chrom_df <- tibble(
  Scan_Index  = seq_along(sps_ms1),
```

```
  RT_sec      = round(rtime(sps_ms1), 2),
  RT_min      = round(rtime(sps_ms1) / 60, 2),
  TIC         = round(tic(sps_ms1)),
  BPI         = round(sps_ms1$basePeakIntensity),
  BasePeak_MZ = round(sps_ms1$basePeakMZ, 4)
)

cat("Survey Scan Chromatographic Data (First 5 scans):\n")
print(as.data.frame(head(chrom_df, 5)))
```

```
Survey Scan Chromatographic Data (First 5 scans): Scan_Index RT_sec RT_min TIC BPI BasePeak_MZ 1 1 2727.07 45.45
22865070080 2962318336 624.8348 2 2 2730.15 45.50 18657597440 609598400 589.6503 3 3 2734.08 45.57 29984956416
2811934464 667.8817 4 4 2737.12 45.62 39212593152 5434890752 667.8817 5 5 2740.27 45.67 41834815488 6174048256
667.8816
```

5.6.4 Listing 5.4: Lazy Data Processing and Peak Thresholding

R Lab Listing 4: Listing 5.4: Transient Noise Removal via Processing Queue [R 4.4 / Bioconductor]

```
# Extract MS2 spectra
sps_ms2 <- filterMsLevel(sps_ondisk, msLevel = 2L)

# Apply noise filtering to fragment peaks
sps_ms2_clean <- filterIntensity(sps_ms2, intensity = c(100, Inf))

# Evaluate first scan before and after queue execution
pks_before <- peaksData(sps_ms2[1])[[1]]
pks_after  <- peaksData(sps_ms2_clean[1])[[1]]

cat(sprintf("MS2 Scan 1 (Acquisition %d):\n", acquisitionNum(sps_ms2[1])))
cat(sprintf("  Raw Peaks:     %d\n", nrow(pks_before)))
cat(sprintf("  Cleaned Peaks: %d (>= 100 counts)\n", nrow(pks_after)))
```

```
MS2 Scan 1 (Acquisition 21946): Raw Peaks: 450 Cleaned Peaks: 450 (>= 100 counts)
```

5.7 Five Deadly Sins of Mass Spectrometry Raw Data Ingestion

1. **Premature Memory Materialization:** Loading an entire cohort of raw spectral arrays into RAM using `MsBackendMemory` causes memory exhaustion. Always maintain large cohorts under `MsBackendMzR` or `MsBackendHdf5Peaks`.

2. **Improper Filter Order in msconvert:** Applying a thresholding filter prior to vendor centroiding strips profile baseline envelopes, causing vendor DLLs to fail and abort centroiding.

3. **Omitting Index Generation:** Producing unindexed `mzML` files forces parsers to sequentially scan multi-gigabyte XML text trees from byte zero to locate late-eluting spectra, slowing random access from milliseconds to minutes.

4. **Neglecting Acquisition Mode Verification:** Processing DIA isolation window spectra with DDA single-precursor search engines silently attributes chimeric fragment arrays to single parent ions, corrupting identification statistics.

5. **Unchecked Re-Centroiding:** Passing already centroided vendor data through a secondary centroiding algorithm splits true ion bars into artificial clusters, distorting isotopic ratios.

5.8 Pre-Flight Diagnostic Checklist

Table 5.2 provides an audit checklist to execute on all incoming files prior to feature detection or database searching.

Table 5.2: Diagnostic checklist for raw mass spectrometry data ingestion.

Category	Diagnostic Check	Acceptance Criteria	Status
Schema	File wrapped in `<indexedmzML>` container.	Seek offset table exists at file footer.	[ ]
Centroiding	Spectra converted via vendor peak-picking.	Discrete $(m/z, I)$ bars; zero profile baseline.	[ ]
Precision	Binary arrays encoded with 32-bit floating points.	Adequate dynamic range; 50% file size reduction.	[ ]
Backend	Multi-file cohort loaded under on-disk backend.	RAM allocation $\leq$ 1 MB per raw file handle.	[ ]
Duty Cycle	Cycle duration between MS1 scans tracked.	Stable cycle timing consistent with TopN protocol.	[ ]
Isolation	Precursor isolation windows audited.	Narrow windows ($\leq$ 1.5 Da) for DDA; broad for DIA.	[ ]

5.9 Chapter Exercises

Exercise 5.1 (Profile versus Centroid Storage Dynamics). An Orbitrap instrument operates at 60,000 resolving power, detecting approximately $2,500$ distinct ion features per full MS1 scan. In profile mode, each peak is represented by an average of 18 discrete sampling points. In centroid mode, each peak is reduced to a single $(m/z, \text{intensity})$ coordinate. Numeric coordinates are stored in 64-bit double precision (8 bytes per value).

1. Calculate the raw binary data volume (in kilobytes) required to store a single MS1 scan in profile mode and centroid mode.

2. Compute the compression factor achieved by centroiding and state the pedagogical trade-off between profile and centroid representations.

Exercise 5.2 (Data-Dependent Duty Cycle and Survey Timing). In an LC-MS/MS experiment recorded in `MS3TMT11.mzML`, 30 MS1 survey scans span retention times from $2,727.07$ to $2,826.18$ seconds.

1. Calculate the mean duration of the duty cycle between consecutive MS1 scans.

2. Determine the maximum number of dependent scans triggered per survey cycle and classify the acquisition protocol.

Exercise 5.3 (RAM Scalability: Cohort Analysis). A single LC-MS file allocates 479.99 KB under `MsBackendMzR` and 10,295.07 KB under `MsBackendMemory`.

1. Compute the required RAM for cohorts of 10, 50, 200, and 1,000 files under both backends.

2. Explain the computational failure mode when processing clinical cohorts under an in-memory paradigm.

Exercise 5.4 (Random Access Seeking Complexity). A 2 GB `mzML` file contains 15,000 spectra. A researcher requires access to scan number 10,000.

1. Contrast the algorithmic time complexity of retrieving scan 10,000 from an unindexed `mzML` file versus an indexed `<indexedmzML>` container.

2. Estimate the physical I/O latency difference between the two access patterns.

5.10 Worked Solutions to Chapter Exercises

5.10.1 Solution to Exercise 5.1: Profile versus Centroid Storage Dynamics

Part 1: Binary Data Volume Calculation Each data point requires 16 bytes (8 bytes for m/z plus 8 bytes for intensity).

- Profile mode: $2,500$ peaks $\times$ 18 points/peak $\times$ 16 bytes/point $= 720,000$ bytes ≈ 703.12 KB.
- Centroid mode: $2,500$ peaks $\times$ 1 point/peak $\times$ 16 bytes/point $= 40,000$ bytes ≈ 39.06 KB.

Part 2: Compression Factor and Trade-off The compression ratio is:

$$\frac{703.12 \text{ KB}}{39.06 \text{ KB}} = 18.0\times \tag{5.3}$$

Centroiding yields an 18-fold reduction in data volume. The trade-off is the irreversible loss of raw peak shape, baseline noise distributions, and peak asymmetry information. Profile mode is required for evaluating mass analyzer tuning and baseline signal-to-noise models; centroid mode is mandatory for search engine matching efficiency and manageable storage footprints.

5.10.2 Solution to Exercise 5.2: Data-Dependent Duty Cycle and Survey Timing

Part 1: Mean Duty Cycle Duration Across the 29 completed MS1 cycles spanning $2,727.07$ to $2,826.18$ seconds, the mean cycle duration is:

$$\bar{\Delta t}_{\text{cycle}} = \frac{2,826.18 - 2,727.07}{29} = 3.36 \text{ seconds} \quad (\text{SD} = 0.36 \text{ s}, \text{Range: } [2.99, 3.99] \text{ s}) \tag{5.4}$$

Part 2: Dependent Scan Count and Protocol Classification The mean number of dependent MS2 scans acquired per cycle is 16.3, with a maximum of 20 dependent scans occurring during high-complexity chromatographic elution. This confirms a Top20 Data-Dependent Acquisition (DDA) method, where the mass spectrometer interrogates up to the 20 most abundant precursors detected in each survey scan.

Table 5.3: Memory scalability across study cohort sizes.

Cohort Size (Files)	RAM On-Disk (MB)	RAM In-Memory (MB)	RAM In-Memory (GB)
10	4.69	100.54	0.10
50	23.44	502.69	0.49
200	93.75	2,010.76	1.96
1,000	468.74	10,053.78	9.82

5.10.3 Solution to Exercise 5.3: RAM Scalability: Cohort Analysis

Part 1: RAM Scaling Table Using verified per-file footprints (0.469 MB on-disk vs 10.054 MB in-memory):

Part 2: Failure Mode in Clinical Cohorts In standard full-length LC-MS runs (which contain 30,000--60,000 scans per file rather than the 994 scans in this benchmark chunk), memory consumption increases by roughly 40-fold. An in-memory representation of 1,000 full-length files requires approximately 400 GB of RAM, triggering an out-of-memory error on standard compute nodes. The on-disk backend requires under 19 GB of RAM for the same cohort, preserving operational stability.

5.10.4 Solution to Exercise 5.4: Random Access Seeking Complexity

Part 1: Algorithmic Complexity

- **Unindexed mzML:** $O(N)$ linear time complexity. The XML parser must scan sequentially from byte zero, reading and decoding every opening and closing tag across $9,999$ preceding spectra before reaching scan $10,000$.
- **Indexed mzML:** $O(1)$ constant time complexity. The parser reads the byte offset table from the document footer, identifies the memory address for scan $10,000$, and issues a direct system `fseek()` call to that location.

Part 2: Physical Latency Difference In an unindexed 2 GB file, scanning through 1.9 GB of XML text requires disk reads and string tokenization taking approximately 8.5 seconds. In an indexed file, seeking directly to the byte address executes in under 0.2 milliseconds, an acceleration factor exceeding $40,000\times$.

Bibliography

[1] Martens, L., Chambers, M., Hermjakob, H., et al.: mzML---a community standard for mass spectrometry data. *Molecular & Cellular Proteomics* **10**(1), R110.000133 (2011). doi:10.1074/mcp.R110.000133

[2] Pedrioli, P.G.A., Eng, J.K., Hubley, R., et al.: A common open representation of mass spectrometry data and its application in a modular software pipeline. *Nature Biotechnology* **22**(11), 1459--1466 (2004). doi:10.1038/nbt1031

[3] Deutsch, E.W.: The mzML open mass spectrometry data standard. *Experimental Review of Proteomics* **7**(2), 179--181 (2010). doi:10.1586/epr.09.108

[4] Chambers, M.C., Maclean, B., Burke, R., et al.: A cross-platform toolkit for mass spectrometry and proteomics. *Nature Biotechnology* **30**(10), 918--920 (2012). doi:10.1038/nbt.2377

[5] Bhamber, R.S., Jankevics, A., Deutsch, E.W., Jones, A.R., Dowsey, A.W.: mzMLb: A versatile, modern, and open binary mass spectrometry data format. *Journal of Proteome Research* **20**(1), 172--183 (2021). doi:10.1021/acs.jproteome.0c00192

[6] Schramm, T., Hester, A., Klinkert, I., et al.: imzML---a common data format for the flexible exchange and processing of mass spectrometry imaging data. *Journal of Proteomics* **75**(16), 5106--5110 (2012). doi:10.1016/j.jprot.2012.07.026

[7] Hulstaert, N., Shofstahl, J., Sachsenberg, T., et al.: ThermoRawFileParser: modular, scalable, and cross-platform RAW file conversion. *Journal of Proteome Research* **19**(1), 537--542 (2020). doi:10.1021/acs.jproteome.9b00328

[8] Gatto, L., Lilley, K.S.: MSnbase---an R/Bioconductor package for isobaric tagged and label-free mass spectrometry data processing. *Bioinformatics* **28**(2), 288--289 (2012). doi:10.1093/bioinformatics/btr645

[9] Gatto, L., Rainer, J., Gibb, S.: MSnbase, Spectra, and QFeatures: Modern mass spectrometry data structures in Bioconductor. *Journal of Proteome Research* **20**(1), 1063--1069 (2021). doi:10.1021/acs.jproteome.0c00864

[10] Rainer, J., Vicini, A., Salzer, L., et al.: A modular and expandable ecosystem for metabolomics data annotation in R. *Metabolites* **12**(2), 173 (2022). doi:10.3390/metabo12020173

[11] Huber, W., Carey, V.J., Gentleman, R., et al.: Orchestrating high-throughput genomic analysis with Bioconductor. *Nature Methods* **12**(2), 115--121 (2015). doi:10.1038/nmeth.3252

[12] Gentleman, R.C., Carey, V.J., Bates, D.M., et al.: Bioconductor: open software development for computational biology and bioinformatics. *Genome Biology* **5**(10), R80 (2004). doi:10.1186/gb-2004-5-10-r80

[13] Eddelbuettel, D., François, R.: Rcpp: Seamless R and C++ integration. *Journal of Statistical Software* **40**(8), 1--18 (2011). doi:10.18637/jss.v040.i08

[14] Fischer, B., Pau, G., Smith, M.: *rhdf5: R Interface to HDF5*. R package version 2.40.0 (2020). doi:10.18129/B9.bioc.rhdf5

[15] Tautenhahn, R., Böttcher, C., Neumann, S.: Highly sensitive feature detection for high resolution LC/MS. *BMC Bioinformatics* **9**, 504 (2008). doi:10.1186/1471-2105-9-504

[16] Cox, J., Mann, M.: MaxQuant enables high peptide identification rates, individualized p.p.b.-range mass accuracies and proteome-wide protein quantification. *Nature Biotechnology* **26**(12), 1367--1372 (2008). doi:10.1038/nbt.1511

[17] Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., et al.: The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data* **3**, 160018 (2016). doi:10.1038/sdata.2016.18

[18] Deutsch, E.W., Bandeira, N., Sharma, V., et al.: The ProteomeXchange consortium in 2020: enabling 'big data' approaches in proteomics. *Nucleic Acids Research* **48**(D1), D1145--D1152 (2020). doi:10.1093/nar/gkz984